

Problem Set 1

BM2043 - Algorithms and Data Structure Lab

Basic Recursion

Problem 1

Abhishek is studying the growth of a certain species of rabbits in his biology class. The population of these rabbits at any given month can be described using a sequence similar to the Fibonacci sequence, where the number of rabbits each month is the sum of the previous two months, starting with one pair of rabbits.

This phenomenon starts with a single pair of rabbits in the first month (considered as month 1), and none before that. By month 2, the pair matures and by month 3, they produce another pair, and so on. The sequence begins as follows: 0 pairs in month 0, 1 pair in month 1, 1 pair in month 2, 2 pairs in month 3, and so forth.

Given a number n representing the month, help Abhishek calculate how many pairs of rabbits will be there at the end of the month n .

Example 1:

- **Input:** $n = 2$
- **Output:** 1
- **Explanation:** At the end of month 2, there is 1 pair of rabbits (one mature pair).

Example 2:

- **Input:** $n = 3$
- **Output:** 2
- **Explanation:** At the end of month 3, there are 2 pairs of rabbits (one mature pair and one new pair).

Example 3:

- **Input:** $n = 4$
- **Output:** 3
- **Explanation:** At the end of month 4, there are 3 pairs of rabbits (two mature pairs and one new pair).

Constraints:

- $0 \leq n \leq 30$

Class Definition:

```
class Solution {
public:
    int function(int n) {
        // write your code here
    }
};
```

Problem 2

Anu, a biomedical researcher, is conducting experiments on different combinations of chemical compounds. For each experiment, she can use any compound exactly once in a specific sequence. Anu needs to determine how many unique sequences of applying these compounds are possible given that she has N different compounds to work with.

The number of possible sequences in which she can arrange these compounds is given by the factorial of N . Help Anu calculate the number of different ways she can sequence her experiments with the given compounds.

Example 1:

- **Input:** $N = 5$
- **Output:** 120
- **Explanation:** Anu has 5 different compounds. The number of ways she can sequence them in her experiments is $5 \times 4 \times 3 \times 2 \times 1 = 120$.

Example 2:

- **Input:** $N = 4$
- **Output:** 24
- **Explanation:** Anu has 4 different compounds. The number of ways she can sequence them is $4 \times 3 \times 2 \times 1 = 24$.

Constraints:

- $0 \leq N \leq 18$

Class Definition:

```
class Solution {
public:
    long long int possible_sequences(int N) {
        // code here
    }
};
```

Searching

Problem Statement 1

Abhay, a student at IIT Hyderabad, is volunteering at the Knowledge Resource Center. He is tasked with organizing a shelf that contains numerous academic journals. Each journal has a unique identification number. Today, Abhay needs to find a particular journal that his professor urgently requires for research.

Given a list of journal identification numbers and the specific journal ID his professor needs, help Abhay determine the position of this journal on the shelf. Use a linear search, as the journals are not in any specific order.

Example 1:

- **Input:** IDs = [101, 234, 145, 876, 321], Target ID = 145
- **Output:** 2
- **Explanation:** The journal with ID 145 is at the 3rd position in the list (index starts at 0).

Example 2:

- **Input:** IDs = [900, 445, 123, 345], Target ID = 100
- **Output:** -1
- **Explanation:** The journal with ID 100 is not present in the list.

Constraints:

- $1 \leq \text{number of journals} \leq 1000$
- Each journal ID is a unique positive integer.

Class Definition:

```
class Solution {
public:
    long long int findJournalIndex(vector<int>& journalIDs, int targetID) {
        // code here
    }
};
```

Problem Statement 2

Pritesh, a researcher in the Biomedical Department, is cataloging a large collection of medical research papers by their reference numbers, which are all sorted in ascending order. Today, Pritesh needs to quickly find a specific research paper required for a critical review on an emerging medical treatment.

Given a sorted list of reference numbers of the research papers and the specific reference number Pritesh is searching for, help him determine the position of this paper using binary search.

Example 1:

- **Input:** Reference Numbers = [100, 150, 200, 250, 300, 350], Target Reference = 250
- **Output:** 3
- **Explanation:** The paper with reference number 250 is at the 4th position in the array (index starts at 0).

Example 2:

- **Input:** Reference Numbers = [102, 112, 123, 133, 143, 153], Target Reference = 120
- **Output:** -1
- **Explanation:** The paper with reference number 120 is not present in the list.

Constraints:

- $1 \leq \text{number of papers} \leq 100000$
- Each reference number is a unique positive integer.

Hint: Since the array is sorted and the number of papers is of the order 10^5 , we should use binary search!!

Class Definition:

```
class Solution {
public:
    long long int findPaperIndex(vector<int>& references, int targetReference) {
        // code here
    }
};
```

Problem Statement 3

Arsh is at the Tinkerer Lab at IIT Hyderabad, where he is experimenting with a sequence of sensor readings that are represented as integers in an array. For his project, Arsh needs to find the most frequently occurring sensor reading from this array.

Given an array of sensor readings, help Arsh determine the frequency of the most common reading.

Example 1:

- **Input:** Readings = [1, 2, 2, 3, 3, 3]
- **Output:** 3
- **Explanation:** The number '3' appears the most frequently, three times.

Example 2:

- **Input:** Readings = [4, 4, 5, 6, 6, 6]
- **Output:** 3
- **Explanation:** The number '6' appears three times, which is the highest frequency in this array.

Constraints:

- $1 \leq \text{size of array} \leq 1000$
- Each sensor reading is a positive integer.

Class Definition:

```
class Solution {
public:
    int maxFrequency(vector<int>& readings) {
        // code here
    }
};
```

Sorting

Problem Statement

In a boutique spice shop in Jaipur, Pranav is tasked with organizing an extensive collection of spice jars based on their weight. Each jar's weight is recorded on a label, but over time, these have been mixed up. For an upcoming inventory review, Pranav needs to sort all the spice jars in ascending order of weight.

Given an array of the weights of spice jars, help Pranav sort this array using different sorting algorithms. Each sorting method should be implemented in its separate class to allow for a clear comparison of their efficiencies.

Example:

- **Input:** Weights = [45, 22, 12, 8, 31, 28]
- **Output for each sorting method:** [8, 12, 22, 28, 31, 45]

Constraints:

- $1 \leq \text{size of array} \leq 500$
- Weights are positive integers.

Class Definitions:

```
class BubbleSort {
public:
    vector<int> sortArray(vector<int>& weights) {
        // Implement Bubble Sort
    }
};

class InsertionSort {
public:
    vector<int> sortArray(vector<int>& weights) {
        // Implement Insertion Sort
    }
};

class SelectionSort {
public:
    vector<int> sortArray(vector<int>& weights) {
        // Implement Selection Sort
    }
};

class MergeSort {
public:
    vector<int> sortArray(vector<int>& weights) {
        // Implement Merge Sort
    }
};

class QuickSort {
public:
    vector<int> sortArray(vector<int>& weights) {
        // Implement Quick Sort
    }
};
```

Miscellaneous

Problem Statement 1

A software company is developing a new feature for their financial planning app, which involves finding budget combinations from a set of expense options. In this app, a financial analyst named Ishaan has a

list of potential monthly expense adjustments represented as integer values (both increases and decreases). Ishaan's task is to identify any three adjustments that, when combined, will balance to a zero net change, aiding users in maintaining their current budget levels.

Help Ishaan find any triplet of indices from this list of adjustments where the sum equals zero. If such a triplet exists, return the indices; if no such triplet exists, return $[-1, -1, -1]$.

Example 1:

- **Input:** adjustments = $[-500, 200, 300, -200, 100]$
- **Output:** $[0, 1, 3]$
- **Explanation:** The adjustments at indices 0, 1, and 3 add up to zero ($-500 + 200 - 200 = 0$).

Example 2:

- **Input:** adjustments = $[100, -150, 50, 200, -100]$
- **Output:** $[0, 2, 4]$
- **Explanation:** The adjustments at indices 0, 2, and 4 sum up to zero ($100 + 50 - 150 = 0$).

Example 3:

- **Input:** adjustments = $[200, 300, 500]$
- **Output:** $[-1, -1, -1]$
- **Explanation:** No triplet of indices has a sum of zero.

Constraints:

- $3 \leq \text{adjustments.length} \leq 3000$
- $-10^5 \leq \text{adjustments}[i] \leq 10^5$

Class Definition:

```
class Solution {
public:
    vector<int> findBalancingTriplets(vector<int>& adjustments) {
        // Implement a method to find three indices with a sum of zero
    }
};
```

Problem Statement 2 (Kadane's Algorithm)

Arin is working on a financial analytics project at her company. She has a list representing the monthly profit changes of a startup over several months, where positive numbers indicate gains and negative numbers indicate losses. Arin needs to identify the continuous period (sequence of months) that maximizes the net profit gain to advise on financial strategies.

Given an array of integers representing these monthly changes in profit, help Arin determine the maximum sum of any contiguous subarray.

Example 1:

- **Input:** `nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]`
- **Output:** 6
- **Explanation:** The subarray `[4, -1, 2, 1]` has the largest sum of 6, which represents the best continuous profit period.

Example 2:

- **Input:** `nums = [1]`
- **Output:** 1
- **Explanation:** The only month shows a profit of 1.

Example 3:

- **Input:** `nums = [5, 4, -1, 7, 8]`
- **Output:** 23
- **Explanation:** The subarray `[5, 4, -1, 7, 8]` has the largest sum of 23, which indicates the most profitable sequence of months.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Class Definition:

```
class Solution {
public:
    long long int maxSubArraySum(vector<int>& nums) {
        // code here
    }
};
```

Problem Statement 3

Shweta is tasked with overseeing the class representative elections at her college. Each student casts their vote by writing down the candidate's unique number on a ballot. To win the election outright, a candidate must receive more than half of the total votes. Shweta has compiled all the votes and needs to quickly determine the winner.

Given an array `nums` where each element represents a vote for a candidate, help Shweta find the majority element, which is the candidate number that appears more than $\lfloor n/2 \rfloor$ times in the array.

Example 1:

- **Input:** `nums = [3, 2, 3]`
- **Output:** 3
- **Explanation:** Candidate number 3 has received more than half of the total votes, thereby winning the election.

Example 2:

- **Input:** `nums = [2, 2, 1, 1, 1, 2, 2]`
- **Output:** 2
- **Explanation:** Candidate number 2 has received more than half of the total votes, thereby winning the election.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5 \times 10^4$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

Class Definition:

```
class Solution {
public:
    int findMajorityElement(vector<int>& nums) {
        // Hint : Boyer-Moore Voting Algorithm
    }
};
```

Problem Statement 4

In the village of Kandi near IIT Hyderabad, a farmer has set up several stalls along a straight path. The stalls are positioned at different points, and the farmer needs to house his aggressive cows in these stalls. To prevent the cows from fighting, he wants to place them in such a way that the minimum distance between any two cows is as large as possible. Given the positions of the stalls in the form of an array *arr* and the number of aggressive cows *k*, help the farmer determine the maximum possible minimum distance between any two cows.

Example 1:

- **Input:** `arr = [1, 2, 4, 8, 9]`, $k = 3$
- **Output:** 3
- **Explanation:** The cows can be placed at positions 1, 4, and 8, which ensures the minimum distance between any two cows is 3.

Example 2:

- **Input:** `arr = [1, 2, 3, 4, 7]`, $k = 3$
- **Output:** 3
- **Explanation:** The cows can be placed at positions 1, 4, and 7, ensuring a minimum distance of 3 between any two cows.

Constraints:

- $2 \leq n \leq 10^5$
- $1 \leq \text{arr}[i] \leq 10^9$
- $2 \leq k \leq n$

Class Definition:

```
class Solution {
public:
    int aggressiveCows(vector<int>& stalls, int n, int k) {
        // code here
    }
};
```

Problem Statement 5

Abhay and Nisarg are working on a collaborative project that involves data from two different sensors. Each sensor records temperature data at regular intervals, and the data is stored in two sorted arrays. The team needs to find the median temperature from the combined data of both sensors to ensure accurate analysis. However, due to limited processing power, they must find this median efficiently.

Given two sorted arrays `nums1` and `nums2` representing the temperature readings from two sensors, help Abhay and Nisarg find the median of the combined sorted data. The overall runtime complexity should be $O(\log(m + n))$.

Example 1:

- **Input:** `nums1 = [1, 3]`, `nums2 = [2]`
- **Output:** 2.00000
- **Explanation:** The merged array is `[1, 2, 3]`, and the median is 2.

Example 2:

- **Input:** `nums1 = [1, 2]`, `nums2 = [3, 4]`
- **Output:** 2.50000
- **Explanation:** The merged array is `[1, 2, 3, 4]`, and the median is $(2 + 3)/2 = 2.5$.

Constraints:

- `nums1.length == m`
- `nums2.length == n`
- $0 \leq m \leq 1000$
- $0 \leq n \leq 1000$
- $1 \leq m + n \leq 2000$
- $-10^6 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^6$

Class Definition:

```
class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        // Implement the optimized approach here
    }
};
```

Problem Statement 6

Cheeku works at a lost-and-found office where she handles missing items. Recently, she received a list of ticket numbers that have been recovered but not sorted. She needs to find the smallest positive ticket number that is still missing from the recovered list, but she must do it efficiently due to the large number of tickets.

Given an unsorted integer array `nums` representing the recovered ticket numbers, help Cheeku find the smallest positive integer that is missing from the list. The solution should run in $O(n)$ time and use $O(1)$ auxiliary space.

Example 1:

- **Input:** `nums = [1, 2, 0]`
- **Output:** 3
- **Explanation:** The ticket numbers in the range $[1, 2]$ are all present in the array.

Example 2:

- **Input:** `nums = [3, 4, -1, 1]`
- **Output:** 2
- **Explanation:** Ticket number 1 is present, but ticket number 2 is missing.

Example 3:

- **Input:** `nums = [7, 8, 9, 11, 12]`
- **Output:** 1
- **Explanation:** The smallest positive ticket number 1 is missing.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

Class Definition:

```
class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {
        // Implement the optimal approach here
    }
};
```